Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Βάσεις Δεδομένων

Εξαμηνιαία Εργασία Έτους 2025-2026

Ίωνας Τσακίρης 03123077*- Ομάδα 77

13 Μαΐου 2026

*el23077@mail.ntua.gr

Περιεχόμενα

1 Εισαγωγή

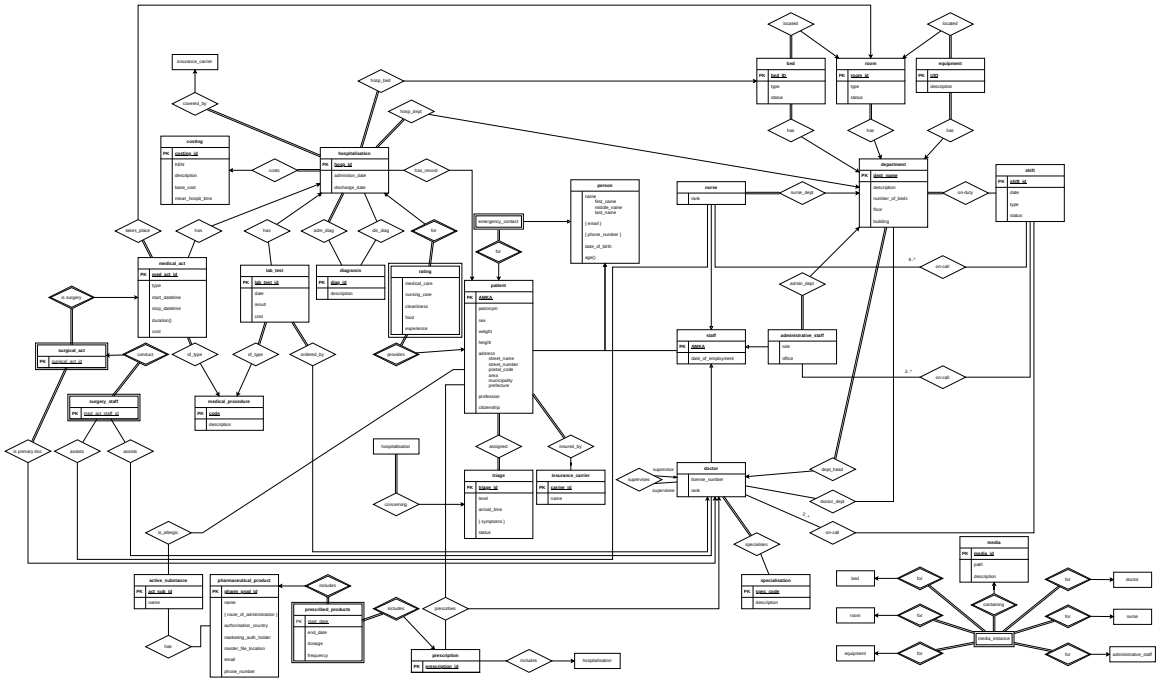
Η παρούσα αναφορά σκιαγραφεί την ανάλυση και υλοποίηση μιας βάσης δεδομένων στο πλαίσιο του μαθήματος «Βάσεις Δεδομένων» του 6ου εξαμήνου της Σχολής ΗΜΜΥ, ΕΜΠ.

Σκοπός μας είναι να μοντελοποιήσουμε τις διάφορες πληροφοριακές ανάγκες ενός νοσοκομείου, ανάγοντάς τες σε ένα σύνολο σχέσεων και οντοτήτων, το οποίο διαχειριζόμαστε υπολογιστικά. Συγκεκριμένα, υλοποιούμε μια σχεσιακή βάση, χρησιμοποιώντας το σύστημα διαχείρισης βάσεων δεδομένων MySQL/MariaDB. Επιπλέον, αναγνωρίζουμε και επιβάλλουμε διάφορους περιορισμούς στα δεδομένα, είτε λογικούς, είτε ως λειτουργική απαίτηση του νοσοκομείου, με χρήση κατάλληλων `constraints` και `triggers`. Τέλος, ελέγχουμε την ορθότητα της υλοποίησής μας θέτοντας έναν αριθμό ερωτήσεων προς τη βάση, λαμβάνοντας κάθε φορά ανάλογες απαντήσεις.

Σημειώνουμε ότι, πέραν των αρχείων SQL που ορίζουν και αλληλεπιδρούν με τη βάση, στον φάκελο `code/`, προσφέρουμε ένα αρχείο Python το οποίο δημιουργεί δεδομένα για τη βάση με ημι-τυχαίο τρόπο, καθώς και ένα bash script που εκτελεί το σύνολο των προαναφερθέντων ερωτημάτων προς τη βάση, και καταγράφει τα αποτέλεσματά τους.

2 ERD

Το διάγραμμα οντοτήτων-συσχετίσεων της βάσης δημιουργήθηκε με τη βοήθεια του draw.io[1].



Σχήμα 1: Διάγραμμα ER βάσης

2.1 Relations

- ασθενής **είναι** άτομο

[1] `https://www.drawio.com/`

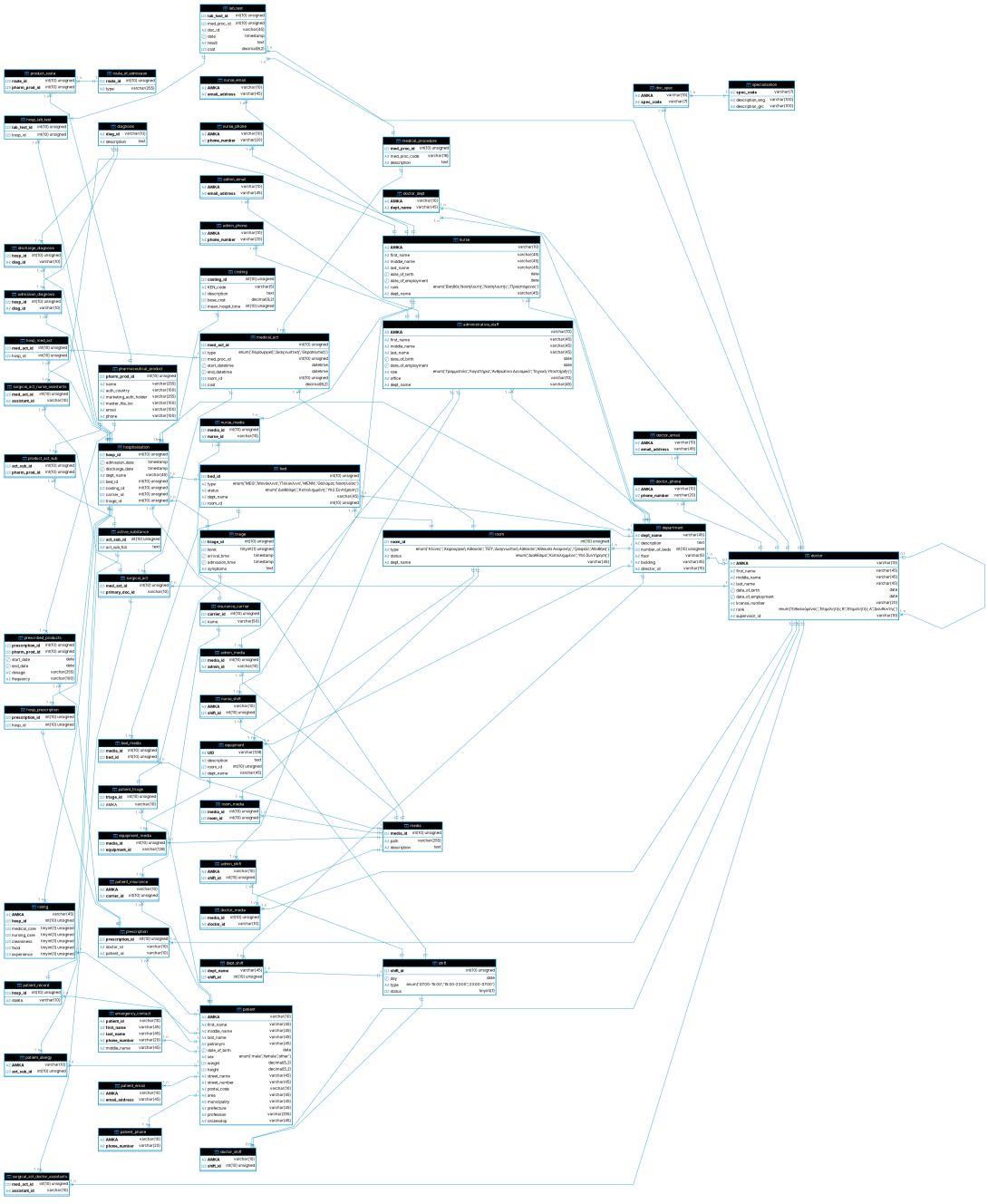
- προσωπικό **είναι** άτομο
- διοικητικό προσωπικό **είναι** προσωπικό
- νοσηλευτής **είναι** προσωπικό
- γιατρός **είναι** προσωπικό
- επόπτης **είναι** γιατρός
- γιατρός **μπορεί να έχει έναν** επόπτη
- γιατρός **ανήκει σε** $>= 1$ τμήματα
- γιατρός **έχει** $>= 1$ ειδικότητες
- **κάθε** νοσηλευτής **ανήκει σε ένα** τμήμα
- **κάθε** τμήμα **έχει έναν** διευθυντή (γιατρό)
- **κάθε** τμήμα **έχει** δωμάτια
- **κάθε** τμήμα **έχει** κλίνες
- **κάθε** τμήμα **μπορεί να έχει** εξοπλισμό
- **κάθε** κλίνη **βρίσκεται σε** δωμάτιο
- **κάθε** εξοπλισμός **βρίσκεται σε** δωμάτιο
- **κάθε** τμήμα **έχει** βάρδιες
- **κάθε** βάρδια **απασχολεί έναν αριθμό** γιατρών
- **κάθε** βάρδια **απασχολεί έναν αριθμό** νοσηλευτών
- **κάθε** βάρδια **απασχολεί έναν αριθμό** διοικητικού προσωπικού
- **κάθε** ασθενής **έχει** $>=1$ ασφαλιστικό φορέα
- **κάθε** ασθενής **έχει** $>=1$ καταθέσεις διαλογής (στο ιστορικό του)
- ασθενής **μπορεί να έχει** $>=1$ νοσηλείες (στο ιστορικό του)
- ασθενής **που χρειάζεται** νοσηλεία **εισάγεται σε** τμήμα **με** κλίνη
- **κάθε** νοσηλεία **έχει** κοστολόγηση
- **κάθε** νοσηλεία **έχει** διάγνωση εισαγωγής
- νοσηλεία **μπορεί να έχει** διάγνωση εξόδου
- νοσηλεία **μπορεί να έχει** αξιολόγηση **από** ασθενή
- νοσηλεία **μπορεί να συνδέεται με** $>= 1$ φαρμακευτικές συνταγές
- νοσηλεία **μπορεί να συνδέεται με** $>= 1$ εργαστηριακές εξετάσεις
- νοσηλεία **μπορεί να συνδέεται με** $>= 1$ ιατρικές επεμβάσεις
- εργαστηριακές εξετάσεις **είναι** ιατρικές πράξεις
- ιατρικές επεμβάσεις **είναι** ιατρικές πράξεις
- χειρουργικές επεμβάσεις **είναι** ιατρικές επεμβάσεις
- **κάθε** χειρουργική επέμβαση **έχει** κύριο χειρούργο **και** $>= 0$ βοηθούς (γιατρούς ή νοσηλευτές)
- **κάθε** φαρμακευτική συνταγή **εκδίδεται από** γιατρό **για** ασθενή
- **κάθε** φαρμακευτική συνταγή **εμπεριέχει** $>= 1$ φαρμακευτικές αγωγές

- **κάθε** φαρμακευτική αγωγή **αφορά ένα** φαρμακευτικό προϊόν
- **κάθε** φαρμακευτικό προϊόν **εμπεριέχει** $>= 1$ δραστικές ουσίες
- ασθενείς **μπορεί να έχουν** καταγεγραμμένες αλλεργίες **σε** δραστικές ουσίες
- προσωπικό **μπορεί να συνδέεται με** φωτογραφικό υλικό
- δωμάτια **μπορεί να συνδέονται με** φωτογραφικό υλικό
- κλίνες **μπορεί να συνδέονται με** φωτογραφικό υλικό
- εξοπλισμός **μπορεί να συνδέεται με** φωτογραφικό υλικό

Σημειώνουμε ότι για την υλοποίηση των ασθενών και του προσωπικού έχουμε χρησιμοποιήσει ένα σχήμα κληρονομικότητας, όπου κάθε ασθενής και κάθε μέλος προσωπικού είναι άτομο και έχει ορισμένα χαρακτηριστικά. Ακόμη, διαχωρίζουμε τα είδη προσωπικού σε ιατρούς, νοσηλευτές, και διοικητικό προσωπικό. Ακολουθούμε, δηλαδή, μια σχεδίαση από πάνω προς τα κάτω.

3 Relational Diagram

Το σχεσιακό διάγραμμα της βάσης δημιουργήθηκε με τη βοήθεια του DBeaver[2], βάσει του DDL που παρατίθεται στην ενότητα 6.



Σχήμα 2: Σχεσιακό διάγραμμα βάσης

[2] https://dbeaver.io/

4 Χαρακτηριστικά Σχήματος

4.1 Constraints

- **Απαγορεύεται η κυκλική αλυσίδα εποπτείας**.
- **Κάθε** ειδικευόμενος **έχει έναν** επόπτη.
- Διευθυντές **δεν έχουν** επόπτη.
- Βάρδιες **χωρίζονται σε** πρωινές, απογευματινές και νυχτερινές.
- **Κάθε** βάρδια **κάθε** τμήματος **έχει** $>= 3$ γιατρούς **και** $>=6$ νοσηλευτές **και** $>= 2$ διοικητικούς υπαλλήλους.
- **Αν σε** βάρδια **υπάρχει** ειδικευόμενος γιατρός **τότε πρέπει να υπάρχει** $>= 1$ γιατρός βαθμίδας Επιμελητής Α΄ **ή** $>= 1$ διευθυντής.
- **Μέγιστο πλήθος βαρδιών:**
  - γιατρός: 15,
  - νοσηλευτής: 20,
  - διοικητικό προσωπικό: 25.
- Προσωπικό δεν μπορεί να έχει διαδοχικές βάρδιες (πρωί **ΚΑΙ** απόγευμα ***Ή*** απόγευμα **ΚΑΙ** νύχτα ***Ή*** νύχτα **ΚΑΙ** πρωί (επόμενης)).
- Προσωπικό **δεν μπορεί να έχει** $>=3$ **διαδοχικές** νυχτερινές βάρδιες.
- Η εξυπηρέτηση διαλογής γίνεται βάσει του επιπέδου επείγοντος με μέθοδο FIFO. Η απαίτηση αυτή υλοποιείται με ένα view.
- **Κάθε τετελεσμένη** νοσηλεία **έχει** διάγνωση εξόδου.
- **Τετελεσμένη** νοσηλεία **μπορεί να έχει** αξιολόγηση.
- Δύο επεμβάσεις **δεν μπορούν να πραγματοποιούνται ταυτόχρονα στον ίδιο** χώρο **και με το ίδιο** προσωπικό.
- **Απαγορεύεται** η συνταγογράφηση φαρμάκων **με** δραστική ουσία **στην οποία είναι αλλεργικός** ο ασθενής.
- (γιατρός, ασθενής, φάρμακο, ημερομηνίας έναρξης) **είναι superkey**.
- Οποιοδήποτε ζεύγος ημερομηνιών (αρχή, τέλος) πρέπει να έχει «τέλος» μετά την «αρχή» ή κενό.
- Οι ιατρικές επεμβάσεις καλύπτονται στις κατηγορίες Α, Β (id < 6609) της δοθείσας κωδικοποίησης[3].
- Οι εργαστηριακές εξετάσεις καλύπτονται στις κατηγορίες Γ, Δ, Ε (id ≥ 6609) της δοθείσας κωδικοποίησης[4].

4.2 Υποθέσεις και Σημειώσεις

- Στην περίπτωση που μια εισαγωγή διαλογής οδηγήσει σε νοσηλεία, θεωρούμε ως ώρα εξυπηρέτησης της διαλογής την ώρα έναρξης της νοσηλείας.
- Κάθε τμήμα διαθέτει αίθουσες με κάποιο τύπο (χειρουργικές, επεμβάσεων, κ.λπ), κατάσταση και μοναδική αρίθμηση.
- Τα τμήματα διαθέτουν εξοπλισμό εντός αιθουσών του τμήματος.
- Η διεύθυνση ασθενών εμπεριέχει τα στοιχεία του ΕΜΕπ

[3] https://www.moh.gov.gr/articles/health/domes-kai-draseis-gia-thn-ygeia/kwdikopoihseis/kleista-enopoihmena-noshlia/713-kwdikopoihseis?fdl=193

[4] βλ. σχόλιο 3

- Χάριν απλότητας της υλοποίησης, θεωρούμε ότι κάθε κοστολόγηση μπορεί να συνδέεται με το πολύ έναν ασφαλιστικό φορέα. Στην περίπτωση, δηλαδή, παράλληλης ασφάλισης του ασθενούς, μόνο ένας φορέας καλύπτει κάθε κοστολόγηση.
- Οι εικόνες αναπαρίστανται ως paths σε υποθετικές πληροφορίες στον δίσκο.
- Επιταχύνουμε την εισαγωγή δεδομένων στο install.sql βάσει του ερωτήματος `A.6`[5].
- Οι ιατρικές ειδικότητες προκύπτουν από την αντίστοιχη λίστα του ΓεΣΥ [6].
- Στο δοθέν αρχείο ΕΛΛΗΝΙΚΗ_ΟΝΟΜΑΤΟΛΟΓΙΑ_ΚΑΙ_ΚΩΔΙΚΟΠΟΙΗΣΗ_ΤΩΝ_ΙΑΤΡΙΚΩΝ_ΠΡΑΞΕΩΝ[7], κάποιες τιμές είναι μη μοναδικές, και παραλείπονται από τη βάση.

Level	Code	Message
Warning	1062	Duplicate entry 'I212243' for key 'med_proc_code'
Warning	1062	Duplicate entry 'I212249' for key 'med_proc_code'
Warning	1062	Duplicate entry 'I212251' for key 'med_proc_code'
Warning	1062	Duplicate entry 'I212243' for key 'med_proc_code'
Warning	1062	Duplicate entry 'I212243' for key 'med_proc_code'
Warning	1062	Duplicate entry 'I212243' for key 'med_proc_code'
Warning	1062	Duplicate entry 'I212243' for key 'med_proc_code'
Warning	1062	Duplicate entry 'I212243' for key 'med_proc_code'
Warning	1062	Duplicate entry 'I212243' for key 'med_proc_code'

- Θεωρούμε ότι η πρόσθετη ημερήσια χρέωση, στην περίπτωση που η παραμονή ενός ασθενούς υπερβεί τον αναμενόμενο χρόνο, ανέρχεται στο 10% του βασικού κόστους.

4.3 Q11

Σημειώνουμε ότι, δεδομένου να μη λάβουμε κενό αποτέλεσμα στο ερώτημα Q11, κάνουμε τις ακόλουθες αλλαγές στα δεδομένα που δίδονται από το `code/insert.sql`:

UPDATE surgical_act
SET primary_doc_id = '0309563392'
WHERE primary_doc_id **IN** ('0509973114', '2312517384', '2103891566', '2402660476');

4.4 Q14

Όμοια με το ερώτημα Q11, κάνουμε τις ακόλουθες αλλαγές στα δεδομένα που δίδονται από το `code/insert.sql`:

UPDATE admission_diagnosis ad
JOIN hospitalisation h **ON** h.hosp_id = ad.hosp_id
SET ad.diag_id = 'A01.4'
WHERE YEAR(h.admission_date) = 2021
LIMIT 10;

UPDATE admission_diagnosis ad
JOIN hospitalisation h **ON** h.hosp_id = ad.hosp_id
SET ad.diag_id = 'A01.4'
WHERE YEAR(h.admission_date) = 2022
LIMIT 10;

UPDATE admission_diagnosis ad
JOIN hospitalisation h **ON** h.hosp_id = ad.hosp_id

[5] `https://dev.mysql.com/doc/workbench/en/workbench-faq.html#faq-workbench-increase-performance`
[6] `https://www.gesy.org.cy/el-gr/annualreport/042026-specialty-restrictions-ahp-mp.xlsx`
[7] βλ. σχόλιο 3

SET ad.diag_id = 'V85.9'
WHERE YEAR(h.admission_date) = 2023
LIMIT 10;

UPDATE admission_diagnosis ad
JOIN hospitalisation h **ON** h.hosp_id = ad.hosp_id
SET ad.diag_id = 'V85.9'
WHERE YEAR(h.admission_date) = 2024
LIMIT 10;

4.5 Χρήση Εργαλείων Τεχνητής Νοημοσύνης

Επιλεγμένα σημεία της εργασίας υλοποιήθηκαν με τη βοήθεια εργαλείων τεχνητής νοημοσύνης. Συγκεκριμένα, χρησιμοποιήθηκε η δωρεάν έκδοση ChatGPT-5.2 στα ακόλουθα σημεία:

- Υλοποίηση triggers για τις σχέσεις εποπτείας μεταξύ ιατρών.
- Υλοποίηση triggers για τους περιορισμούς στην ανάθεση βαρδιών στο προσωπικό.
- Φόρτωση δεδομένων από τη δοθείσα λίστα φαρμακευτικών προϊόντων στη βάση.
- Εύρεση συνολικού μέσου των αξιολογήσεων κάθε ασθενούς για κάθε νοσηλεία στο ιστορικό του, έναντι μέσης αξιολόγησης κάθε νοσηλείας ξεχωριστά (ερώτημα Q06).
- Βελτιστοποίηση λύσης στο ερώτημα Q08.

Οι ανωτέρω «διάλογοι» έχουν μεταφερθεί αυτολεξεί στον φάκελο `docs/transcripts`.

5 Ερωτήματα

Στην παρούσα ενότητα αναλύουμε την πορεία εκτέλεσης των ερωτημάτων Q04 και Q06. Συγκεκριμένα, παραθέτουμε το αντίστοιχο query, το αποτέλεσμα του explain της MySQL, μια εναλλακτική έκδοση με force index και optimizer hints, και, τέλος, συγκρίνουμε τον χρόνο εκτέλεσης των δύο εκδόσεων του κάθε ερωτήματος.

5.1 Q04

SELECT AMKA, **AVG**(medical_care), **AVG**(experience)
FROM (

SELECT lt.doc_id **as** AMKA, medical_care, experience
FROM
    hospitalisation h
    INNER JOIN hosp_lab_test hlt **ON** h.hosp_id = hlt.hosp_id
    INNER JOIN lab_test lt **ON** hlt.lab_test_id = lt.lab_test_id
    INNER JOIN rating r **ON** h.hosp_id = r.hosp_id

UNION ALL

SELECT sa.primary_doc_id **as** AMKA, medical_care, experience
FROM
    hospitalisation h
    INNER JOIN hosp_med_act hma **ON** h.hosp_id = hma.hosp_id
    INNER JOIN surgical_act sa **ON** hma.med_act_id = sa.med_act_id
    INNER JOIN rating r **ON** h.hosp_id = r.hosp_id

UNION ALL

SELECT sada.assistant_id **as** AMKA, medical_care, experience

```
FROM
    hospitalisation   h
    INNER JOIN hosp_med_act hma ON h.hosp_id = hma.hosp_id
    INNER JOIN surgical_act_doctor_assistants sada ON hma.med_act_id = sada.med_act_id
    INNER JOIN rating r ON h.hosp_id = r.hosp_id
) as hlpr
GROUP BY AMKA;
```

5.1.1 Ανάλυση

Ευθύς αμέσως παραθέτουμε το αποτέλεσμα του `EXPLAIN`.

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	PRIMARY	<derived2>	ALL	NULL	NULL	NULL	NULL	436	Using temporary; Using filesort
2	DERIVED	r	ALL	fk_rating_hosp_id	NULL	NULL	NULL	206	
2	DERIVED	h	eq_ref	PRIMARY	PRIMARY	4	ntua_db_2026.r.hosp_id	1	
2	DERIVED	hlt	ref	PRIMARY,fk_hosp_lab_test_hosp_id	fk_hosp_lab_test_hosp_id	4	ntua_db_2026.r.hosp_id	1	Using index
2	DERIVED	lt	eq_ref	PRIMARY	PRIMARY	4	ntua_db_2026.hlt.lab_test_id	1	
3	UNION	sa	index	PRIMARY	fk_surgery_primary_doc_id	42	NULL	101	Using index
3	UNION	hma	eq_ref	PRIMARY,idx_fk_hosp_id	PRIMARY	4	ntua_db_2026.sa.med_act_id	1	
3	UNION	h	eq_ref	PRIMARY	PRIMARY	4	ntua_db_2026.hma.hosp_id	1	
3	UNION	r	ref	fk_rating_hosp_id	fk_rating_hosp_id	4	ntua_db_2026.hma.hosp_id	1	
4	UNION	sada	index	PRIMARY	fk_surg_doc_assist_id	42	NULL	129	Using index
4	UNION	hma	eq_ref	PRIMARY,idx_fk_hosp_id	PRIMARY	4	ntua_db_2026.sada.med_act_id	1	
4	UNION	h	eq_ref	PRIMARY	PRIMARY	4	ntua_db_2026.hma.hosp_id	1	
4	UNION	r	ref	fk_rating_hosp_id	fk_rating_hosp_id	4	ntua_db_2026.hma.hosp_id	1	

Table 1: EXPLAIN output for Q04

Παρατηρούμε ότι το εξωτερικό (primary) select χρησιμοποιεί filesort και προσωρινούς πίνακες για τον υπολογισμό του αποτελέσματος. Συνάμα, το JOIN στον πίνακα rating γίνεται για κάθε γραμμή (ALL στην τέταρτη στήλη του explain). Προς αποφυγήν του τελευταίου, αλλάζουμε τη σειρά με την οποία γίνονται τα JOIN. Επιπλέον, επιβάλλουμε τη χρήση ευρετηρίων στους πίνακες hospitalisation.

```
SELECT AMKA, AVG(medical_care), AVG(experience)
FROM (
    SELECT /*+ JOIN_ORDER(lt, hlt, h, r) */ lt.doc_id as AMKA, medical_care, experience
    FROM hospitalisation h FORCE INDEX (PRIMARY)
    INNER JOIN hosp_lab_test hlt ON h.hosp_id = hlt.hosp_id
    INNER JOIN lab_test lt ON hlt.lab_test_id = lt.lab_test_id
    INNER JOIN rating r ON h.hosp_id = r.hosp_id

    UNION ALL

    SELECT sa.primary_doc_id as AMKA, medical_care, experience
    FROM hospitalisation h FORCE INDEX (PRIMARY)
    INNER JOIN hosp_med_act hma ON h.hosp_id = hma.hosp_id
    INNER JOIN surgical_act sa ON hma.med_act_id = sa.med_act_id
    INNER JOIN rating r ON h.hosp_id = r.hosp_id

    UNION ALL

    SELECT sada.assistant_id as AMKA, medical_care, experience
    FROM hospitalisation h FORCE INDEX (PRIMARY)
    INNER JOIN hosp_med_act hma ON h.hosp_id = hma.hosp_id
    INNER JOIN surgical_act_doctor_assistants sada ON hma.med_act_id = sada.med_act_id
    INNER JOIN rating r ON h.hosp_id = r.hosp_id
) as hlpr
GROUP BY AMKA;
```

5.2 Q06

```
SELECT
    pr. AMKA,
    h. hosp_id,
```

```
    calculate_hospit_cost (h.hosp_id) AS cost,
    ad.diag_id as admission_diagnosis,
    dd.diag_id as discharge_diagnosis ,
    ROUND(
        (r. medical_care + r.nursing_care + r.cleanliness + r.food + r.experience) / 5,
        2) AS avg_rating,
    ROUND(
        AVG((r.medical_care + r.nursing_care + r.cleanliness + r.food + r.experience) / 5) OVER
            (PARTITION BY pr.AMKA),
        2) AS total_avg_rating
FROM patient_record pr
INNER JOIN hospitalisation h ON h.hosp_id = pr.hosp_id
INNER JOIN admission_diagnosis ad ON h.hosp_id = ad.hosp_id
LEFT JOIN discharge_diagnosis dd ON h.hosp_id = dd.hosp_id
LEFT JOIN rating r ON h.hosp_id = r.hosp_id
ORDER BY pr.AMKA;
```

5.2.1 Ανάλυση

Όμοια, με χρήση του `EXPLAIN` λαμβάνουμε:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	ad	index	PRIMARY	fk_adm_diag_diagnosis_id	42	NULL	425	Using index; Using temporary; Using filesort
1	SIMPLE	pr	eq_ref	PRIMARY	PRIMARY	4	ntua_db_2026.ad.hosp_id	1	
1	SIMPLE	h	eq_ref	PRIMARY	PRIMARY	4	ntua_db_2026.ad.hosp_id	1	
1	SIMPLE	dd	ref	PRIMARY	PRIMARY	4	ntua_db_2026.ad.hosp_id	1	
1	SIMPLE	r	ref	fk_rating_hosp_id	fk_rating_hosp_id	4	ntua_db_2026.ad.hosp_id	1	

Table 2: EXPLAIN output for Q06

Μεταβάλλουμε την εκτέλεση αφ΄ ενός επιβάλλοντας τη χρήση index στο patient_record, αφ΄ ετέρου ορίζοντας τη σειρά των JOIN να αρχίζει από το patient_record, καθώς στη συνέχεια ταξινομούμε το τελικό αποτέλεσμα και υπολογίζουμε μέσες τιμές βάσει του pr.AMKA.

```
SELECT /*+ JOIN_ORDER(pr, h, ad, dd, r) */
    pr. AMKA,
    h.hosp_id,
    calculate_hospit_cost (h.hosp_id) AS cost,
    ad.diag_id as admission_diagnosis,
    dd.diag_id as discharge_diagnosis ,
    ROUND((r.medical_care + r.nursing_care + r.cleanliness + r.food + r.experience) / 5, 2) AS
        avg_rating,
    ROUND(AVG((r.medical_care + r.nursing_care + r.cleanliness + r.food + r.experience) / 5)
        OVER (PARTITION BY pr.AMKA), 2) AS total_avg_rating
FROM patient_record pr FORCE INDEX (PRIMARY)
STRAIGHT_JOIN hospitalisation h ON h.hosp_id = pr.hosp_id
STRAIGHT_JOIN admission_diagnosis ad ON h.hosp_id = ad.hosp_id
LEFT JOIN discharge_diagnosis dd ON h.hosp_id = dd.hosp_id
LEFT JOIN rating r ON h.hosp_id = r.hosp_id
ORDER BY pr.AMKA;
```

5.3 Μέτρηση χρόνου

Λαμβάνουμε τον μέσο χρόνο από 10 εκτελέσεις του κάθε query. Τα αποτελέσματα παρατίθενται στον πίνακα 3. Λόγω μικρού πλήθους δεδομένων δε βλέπουμε ουσιώδη διαφορά στον χρόνο εκτέλεσης του Q4. Αντίθετα, για το Q6 έχουμε δελτίωση του μέσω χρόνου μεν, μεγαλύτερη απόκλιση δε.

Μέθοδος / Ερώτημα	Q04	Q06
Απλή κλίση	4.8 ± 1.17ms	22.2 ± 1.47ms
Εναλλακτική έκδοση	5 ± 1.34ms	18.7 ± 5.25ms

Πίνακας 3: Μέσος χρόνος εκτέλεσης queries

6 DDL

Ακολουθεί ο κώδικας MySQL που ορίζει το σχήμα της βάσης δεδομένων.

```
/*
SQL script for schema installation
*/

SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0;
SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS,
    FOREIGN_KEY_CHECKS=0;
SET @OLD_SQL_MODE=@@SQL_MODE,
    SQL_MODE='TRADITIONAL,ALLOW_INVALID_DATES';
-- show count(*) warnings;

DROP SCHEMA IF EXISTS ntua_db_2026;
CREATE SCHEMA ntua_db_2026;
USE ntua_db_2026;


--
-- Table structure for triage
--

CREATE TABLE triage (
    triage_id  INT UNSIGNED NOT NULL AUTO_INCREMENT,
    level TINYINT(1) UNSIGNED NOT NULL CHECK (1 <= level AND level <= 5),
    arrival_time  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    admission_time TIMESTAMP DEFAULT NULL,
    symptoms TEXT NOT NULL,
    PRIMARY KEY (triage_id)
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structure for patient
--

CREATE TABLE patient (
    AMKA VARCHAR(10) NOT NULL,
    first_name  VARCHAR(45) NOT NULL,
    middle_name VARCHAR(45) DEFAULT NULL,
    last_name VARCHAR(45) NOT NULL,
    patronym VARCHAR(45) DEFAULT NULL,
    date_of_birth  DATE NOT NULL,
    sex  ENUM('male', 'female', 'other') NOT NULL,
    weight NUMERIC(5,2) NOT NULL DEFAULT 000.00 CHECK (weight >= 0), -- in kg
    height NUMERIC(5,2) NOT NULL DEFAULT 000.00 CHECK (height >= 0), -- in cm
    -- address
    street_name VARCHAR(45) DEFAULT NULL,
    street_number VARCHAR(45) DEFAULT NULL,
    postal_code VARCHAR(10) DEFAULT NULL,
    area VARCHAR(45) DEFAULT NULL,
```

```
    municipality VARCHAR(45) DEFAULT NULL,
    prefecture VARCHAR(45) DEFAULT NULL,
    -- address end
    profession VARCHAR(255) DEFAULT NULL,
    citizenship VARCHAR(45) DEFAULT NULL,

    PRIMARY KEY (AMKA),
    INDEX idx_patient_last_name (last_name)
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE patient_email (
    AMKA VARCHAR(10) NOT NULL,
    email_address VARCHAR(45) NOT NULL,
    PRIMARY KEY (AMKA, email_address),
    CONSTRAINT fk_patient_email FOREIGN KEY (AMKA) REFERENCES patient (AMKA)
        ON DELETE CASCADE ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE patient_phone (
    AMKA VARCHAR(10) NOT NULL,
    phone_number VARCHAR(20) NOT NULL,
    PRIMARY KEY (AMKA, phone_number),
    CONSTRAINT fk_patient_phone FOREIGN KEY (AMKA) REFERENCES patient (AMKA)
        ON DELETE CASCADE ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE patient_triage (
    AMKA VARCHAR(10) NOT NULL,
    triage_id INT UNSIGNED NOT NULL,
    PRIMARY KEY (triage_id),
    INDEX idx_fk_patient_id (AMKA),
    CONSTRAINT fk_patient_triage_patient_id FOREIGN KEY (AMKA) REFERENCES
        patient (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_patient_triage_triage_id FOREIGN KEY (triage_id) REFERENCES triage
        (triage_id) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structure for emergency contact
--

CREATE TABLE emergency_contact (
    patient_id VARCHAR(10) NOT NULL,
    first_name VARCHAR(45) NOT NULL,
    middle_name VARCHAR(45) DEFAULT NULL,
    last_name VARCHAR(45) NOT NULL,
    phone_number VARCHAR(20) NOT NULL,
    PRIMARY KEY (patient_id, first_name, last_name, phone_number),
    CONSTRAINT fk_emergency_contact FOREIGN KEY (patient_id) REFERENCES patient
        (AMKA) ON DELETE CASCADE ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structure for nurse
--

CREATE TABLE nurse (
    AMKA VARCHAR(10) NOT NULL,
```

```
    first_name VARCHAR(45) NOT NULL,
    middle_name VARCHAR(45) DEFAULT NULL,
    last_name VARCHAR(45) NOT NULL,
    date_of_birth  DATE NOT NULL,
    date_of_employment DATE NOT NULL CHECK (date_of_employment > date_of_birth),
    rank ENUM('Βοηθός-Νοσηλευτή', 'Νοσηλευτής', 'Προϊστάμενος') NOT NULL,
    dept_name VARCHAR(45) NOT NULL,
    PRIMARY KEY (AMKA),
    INDEX idx_nurse_last_name (last_name),
    CONSTRAINT fk_nurse_dept FOREIGN KEY (dept_name) REFERENCES department
        (dept_name) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE nurse_email (
    AMKA VARCHAR(10) NOT NULL,
    email_address VARCHAR(45) NOT NULL,
    PRIMARY KEY (AMKA, email_address),
    CONSTRAINT fk_nurse_email FOREIGN KEY (AMKA) REFERENCES nurse (AMKA)
        ON DELETE CASCADE ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE nurse_phone (
    AMKA VARCHAR(10) NOT NULL,
    phone_number VARCHAR(20) NOT NULL,
    PRIMARY KEY (AMKA, phone_number),
    CONSTRAINT fk_nurse_phone FOREIGN KEY (AMKA) REFERENCES nurse (AMKA)
        ON DELETE CASCADE ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;


--
-- Table structure for administrative staff
--

CREATE TABLE administrative_staff (
    AMKA VARCHAR(10) NOT NULL,
    first_name VARCHAR(45) NOT NULL,
    middle_name VARCHAR(45) DEFAULT NULL,
    last_name VARCHAR(45) NOT NULL,
    date_of_birth  DATE NOT NULL,
    date_of_employment DATE NOT NULL CHECK (date_of_employment > date_of_birth),
    role ENUM('Γραμματεία', 'Λογιστήριο', 'Ανθρώπινο-Δυναμικό', 'Τεχνική-Υποστήριξη') NOT
        NULL,
    office  VARCHAR(10) NOT NULL,
    dept_name VARCHAR(45) NOT NULL,
    PRIMARY KEY (AMKA),
    INDEX idx_admin_last_name (last_name),
    CONSTRAINT fk_admin_dept FOREIGN KEY (dept_name) REFERENCES department
        (dept_name) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE admin_email (
    AMKA VARCHAR(10) NOT NULL,
    email_address VARCHAR(45) NOT NULL,
    PRIMARY KEY (AMKA, email_address),
    CONSTRAINT fk_admin_email FOREIGN KEY (AMKA) REFERENCES
        administrative_staff (AMKA) ON DELETE CASCADE ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
CREATE TABLE admin_phone (
    AMKA VARCHAR(10) NOT NULL,
    phone_number VARCHAR(20) NOT NULL,
    PRIMARY KEY (AMKA, phone_number),
    CONSTRAINT fk_admin_phone FOREIGN KEY (AMKA) REFERENCES
        administrative_staff (AMKA) ON DELETE CASCADE ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;


--
-- Table structure for doctor
--

CREATE TABLE doctor (
    AMKA VARCHAR(10) NOT NULL,
    first_name  VARCHAR(45) NOT NULL,
    middle_name VARCHAR(45) DEFAULT NULL,
    last_name VARCHAR(45) NOT NULL,
    date_of_birth  DATE NOT NULL,
    date_of_employment DATE NOT NULL CHECK (date_of_employment > date_of_birth),
    license_number VARCHAR(20) NOT NULL,
    rank ENUM('Ειδικευόμενος', 'Επιμελητής-Β', 'Επιμελητής-Α', 'Διευθυντής') NOT NULL,
    supervisor_id  VARCHAR(10) NULL,
    PRIMARY KEY (AMKA),
    INDEX idx_doctor_last_name (last_name),
    CONSTRAINT fk_supervisor_id FOREIGN KEY (supervisor_id) REFERENCES doctor
        (AMKA) ON DELETE SET NULL ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE doctor_email (
    AMKA VARCHAR(10) NOT NULL,
    email_address VARCHAR(45) NOT NULL,
    PRIMARY KEY (AMKA, email_address),
    CONSTRAINT fk_doctor_email FOREIGN KEY (AMKA) REFERENCES doctor (AMKA)
        ON DELETE CASCADE ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE doctor_phone (
    AMKA VARCHAR(10) NOT NULL,
    phone_number VARCHAR(20) NOT NULL,
    PRIMARY KEY (AMKA, phone_number),
    CONSTRAINT fk_doctor_phone FOREIGN KEY (AMKA) REFERENCES doctor (AMKA)
        ON DELETE CASCADE ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Doctor specialisation
--

CREATE TABLE specialisation (
    spec_code VARCHAR(7) NOT NULL,
    description_eng  VARCHAR(100) NOT NULL,
    description_grc  VARCHAR(100) NOT NULL,
    PRIMARY KEY (spec_code)
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
CREATE TABLE doc_spec (
    AMKA VARCHAR(10) NOT NULL,
    spec_code VARCHAR(7) NOT NULL,
    PRIMARY KEY (AMKA, spec_code),
    CONSTRAINT fk_doctor_id FOREIGN KEY (AMKA) REFERENCES doctor (AMKA) ON
        DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_spec_id FOREIGN KEY (spec_code) REFERENCES specialisation
        (spec_code) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;



--
-- Table structure for department
--

CREATE TABLE department (
    dept_name VARCHAR(45) NOT NULL,
    description  TEXT NOT NULL,
    number_of_beds INT UNSIGNED NOT NULL DEFAULT 0,
    floor  VARCHAR(5) NOT NULL,
    building  VARCHAR(45) NOT NULL,
    director_id  VARCHAR(10) DEFAULT NULL,
    PRIMARY KEY (dept_name),
    CONSTRAINT fk_dept_head FOREIGN KEY (director_id) REFERENCES doctor (AMKA)
        ON DELETE SET NULL ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structure for doctor-department relation
--

CREATE TABLE doctor_dept (
    AMKA VARCHAR(10) NOT NULL,
    dept_name VARCHAR(45) NOT NULL,
    PRIMARY KEY (AMKA, dept_name),
    CONSTRAINT fk_doc_dept_doctor_id FOREIGN KEY (AMKA) REFERENCES doctor
        (AMKA) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_doc_dept_dept_id FOREIGN KEY (dept_name) REFERENCES
        department (dept_name) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structure for room
--

CREATE TABLE room (
    room_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
    type ENUM('Κλίνες', 'Χειρουργική-Αίθουσα', 'ΤΕΠ', 'Διαγνωστική-Αίθουσα', 'Αίθουσα-Αναμονής',
        'Γραφείο', 'Αποθήκη') NOT NULL,
    status  ENUM('Διαθέσιμο', 'Κατειλημμένο', 'Υπό-Συντήρηση') NOT NULL,
    dept_name VARCHAR(45) NOT NULL,
    PRIMARY KEY (room_id),
    INDEX idx_room_type (type),
    INDEX idx_room_status (status),
    INDEX idx_fk_dept_name (dept_name),
    CONSTRAINT fk_room_dept_id FOREIGN KEY (dept_name) REFERENCES department
        (dept_name) ON DELETE RESTRICT ON UPDATE CASCADE
```

```
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structure for beds
--

CREATE TABLE bed (
    bed_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
    type ENUM('ΜΕΘ', 'Μονόκλινο', 'Πολύκλινο', 'MENN', 'Θάλαμος-Νοσηλείας') NOT NULL,
    status ENUM('Διαθέσιμη', 'Κατειλημμένη', 'Υπό-Συντήρηση') NOT NULL,
    dept_name VARCHAR(45) NOT NULL,
    room_id INT UNSIGNED NOT NULL,
    PRIMARY KEY (bed_id),
    INDEX idx_bed_type (type),
    INDEX idx_bed_status (status),
    INDEX idx_fk_dept_name (dept_name),
    CONSTRAINT fk_bed_dept_id FOREIGN KEY (dept_name) REFERENCES department
        (dept_name) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_bed_room_id FOREIGN KEY (room_id) REFERENCES room (room_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structure for equipment
--

CREATE TABLE equipment (
    UID VARCHAR(128) NOT NULL,
    description  TEXT NOT NULL,
    room_id INT UNSIGNED NOT NULL,
    dept_name VARCHAR(45) NOT NULL,
    PRIMARY KEY (UID),
    UNIQUE (UID, room_id, dept_name),
    INDEX idx_fk_room_id (room_id),
    INDEX idx_fk_dept_name (dept_name),
    CONSTRAINT fk_equip_dept_name FOREIGN KEY (dept_name) REFERENCES
        department (dept_name) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_equip_room_id FOREIGN KEY (room_id) REFERENCES room (room_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structure for shift
--

CREATE TABLE shift (
    shift_id  INT UNSIGNED NOT NULL AUTO_INCREMENT,
    day DATE NOT NULL,
    type  ENUM('07:00–15:00', '15:00–23:00', '23:00–07:00') NOT NULL,
    status BOOLEAN NOT NULL DEFAULT FALSE,
    PRIMARY KEY (shift_id),
    INDEX idx_shift_type (type),
    INDEX idx_shift_day_status_id (day, status, shift_id),        -- used for Q08
    UNIQUE (day, type)
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;


CREATE TABLE nurse_shift (
```

```
    AMKA VARCHAR(10) NOT NULL,
    shift_id  INT UNSIGNED NOT NULL,
    PRIMARY KEY (AMKA, shift_id),
    CONSTRAINT fk_nurse_shift_nurse_id FOREIGN KEY (AMKA) REFERENCES nurse
        (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_nurse_shift_shift_id FOREIGN KEY (shift_id) REFERENCES shift
        (shift_id) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE doctor_shift (
    AMKA VARCHAR(10) NOT NULL,
    shift_id  INT UNSIGNED NOT NULL,
    PRIMARY KEY (AMKA, shift_id),
    CONSTRAINT fk_doctor_shift_doctor_id FOREIGN KEY (AMKA) REFERENCES doctor
        (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_doctor_shift_shift_id FOREIGN KEY (shift_id) REFERENCES shift
        (shift_id) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE admin_shift (
    AMKA VARCHAR(10) NOT NULL,
    shift_id  INT UNSIGNED NOT NULL,
    PRIMARY KEY (AMKA, shift_id),
    CONSTRAINT fk_admin_shift_admin_id FOREIGN KEY (AMKA) REFERENCES
        administrative_staff (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_admin_shift_shift_id FOREIGN KEY (shift_id) REFERENCES shift
        (shift_id) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE dept_shift (
    dept_name VARCHAR(45) NOT NULL,
    shift_id  INT UNSIGNED NOT NULL,
    PRIMARY KEY (dept_name, shift_id),
    CONSTRAINT fk_dept_shift_dept_id FOREIGN KEY (dept_name) REFERENCES
        department (dept_name) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_dept_shift_shift_id FOREIGN KEY (shift_id) REFERENCES shift
        (shift_id) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structure for insurance carrier
--

CREATE TABLE insurance_carrier (
    carrier_id  INT UNSIGNED NOT NULL AUTO_INCREMENT,
    name VARCHAR(50) NOT NULL,
    PRIMARY KEY(carrier_id)
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;


CREATE TABLE patient_insurance (
    AMKA VARCHAR(10) NOT NULL,
    carrier_id  INT UNSIGNED NOT NULL,
    PRIMARY KEY(AMKA, carrier_id),
    CONSTRAINT fk_patient_insurance_patient_id FOREIGN KEY (AMKA) REFERENCES
        patient (AMKA) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_patient_insurance_insurance_id FOREIGN KEY (carrier_id)
        REFERENCES insurance_carrier (carrier_id) ON DELETE RESTRICT ON UPDATE
```

```
        CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structure for costing
--

CREATE TABLE costing (
    costing_id  INT UNSIGNED NOT NULL AUTO_INCREMENT,
    KEN_code VARCHAR(5) NOT NULL,
    description  TEXT,
    base_cost  NUMERIC(9,2) NOT NULL DEFAULT 0.00 CHECK (base_cost >= 0),
    mean_hospit_time INT UNSIGNED NOT NULL DEFAULT 0,
    PRIMARY KEY (costing_id),
    UNIQUE (description)
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structure for dianosis
--     based on ICD-10 codes
--

CREATE TABLE diagnosis (
    diag_id  VARCHAR(10) NOT NULL,
    description  TEXT NOT NULL,
    PRIMARY KEY (diag_id)
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structure for hospitalisation
--

CREATE TABLE hospitalisation (
    hosp_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
    admission_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    discharge_date TIMESTAMP DEFAULT NULL CHECK (discharge_date IS NULL OR
        discharge_date > admission_date),
    dept_name VARCHAR(45) NOT NULL,
    bed_id INT UNSIGNED NOT NULL,
    costing_id  INT UNSIGNED NOT NULL,
    carrier_id  INT UNSIGNED NOT NULL,
    triage_id  INT UNSIGNED NOT NULL,
    PRIMARY KEY (hosp_id),
    INDEX idx_admission_date (admission_date), -- Q09
    INDEX idx_fk_carrier_id (carrier_id),          -- hosp covered by certain carrier
    INDEX idx_fk_dept_name (dept_name),
    CONSTRAINT fk_hosp_dept_id FOREIGN KEY (dept_name) REFERENCES department
        (dept_name) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_hosp_bed_id FOREIGN KEY (bed_id) REFERENCES bed (bed_id) ON
        DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_hosp_carrier_id FOREIGN KEY (carrier_id) REFERENCES
        insurance_carrier (carrier_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_hosp_costing_id FOREIGN KEY (costing_id) REFERENCES costing
        (costing_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_hosp_triage_id FOREIGN KEY (triage_id) REFERENCES triage
        (triage_id) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
--
-- Table structure for relation  hospitalisation  ---< has_record >--> patient
--

CREATE TABLE patient_record (
    AMKA VARCHAR(10) NOT NULL,
    hosp_id INT UNSIGNED NOT NULL,
    PRIMARY KEY (hosp_id),
    INDEX idx_fk_patient_id (AMKA),
    INDEX idx_fk_amka_hosp_id (AMKA, hosp_id), -- Q06
    CONSTRAINT fk_patient_record_patient_id FOREIGN KEY (AMKA) REFERENCES
        patient (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_patient_record_hospitalisation_id FOREIGN KEY (hosp_id)
        REFERENCES hospitalisation (hosp_id) ON DELETE RESTRICT ON UPDATE
        CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structures  for  admission and discharge diagnosis
--

CREATE TABLE admission_diagnosis (
    hosp_id INT UNSIGNED NOT NULL,
    diag_id  VARCHAR(10) NOT NULL,
    PRIMARY KEY (hosp_id, diag_id),
    CONSTRAINT fk_adm_diag_hospitalisation_id FOREIGN KEY (hosp_id) REFERENCES
        hospitalisation (hosp_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_adm_diag_diagnosis_id FOREIGN KEY (diag_id) REFERENCES diagnosis
        (diag_id) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;


CREATE TABLE discharge_diagnosis (
    hosp_id INT UNSIGNED NOT NULL,
    diag_id  VARCHAR(10) NOT NULL,
    PRIMARY KEY (hosp_id, diag_id),
    CONSTRAINT fk_dis_diag_hospitalisation_id FOREIGN KEY (hosp_id) REFERENCES
        hospitalisation (hosp_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_dis_diag_diagnosis_id FOREIGN KEY (diag_id) REFERENCES diagnosis
        (diag_id) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structures  for  medical procedure
--

CREATE TABLE medical_procedure (
    med_proc_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
    med_proc_code VARCHAR(16) NOT NULL,
    description  TEXT NOT NULL,
    PRIMARY KEY (med_proc_id),
    INDEX idx_med_proc_code (med_proc_code),
    UNIQUE (med_proc_code)
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structures  for  lab  test
--
```

```
CREATE TABLE lab_test (
    lab_test_id  INT UNSIGNED NOT NULL AUTO_INCREMENT,
    med_proc_id INT UNSIGNED NOT NULL,
    doc_id VARCHAR(45) NOT NULL,
    date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    result  TEXT NOT NULL,
    cost  NUMERIC(8,2) NOT NULL DEFAULT 0.00 CHECK (cost >= 0),
    PRIMARY KEY (lab_test_id),
    INDEX idx_fk_med_proc_id (med_proc_id),
    INDEX idx_fk_doctor_id (doc_id),
    CONSTRAINT fk_lab_test_med_procedure_id FOREIGN KEY (med_proc_id) REFERENCES
        medical_procedure (med_proc_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_lab_test_doctor_id FOREIGN KEY (doc_id) REFERENCES doctor
        (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;


--
-- Table structure for relation  hospitalisation  <--< has >--- lab_test
--

CREATE TABLE hosp_lab_test (
    lab_test_id  INT UNSIGNED NOT NULL,
    hosp_id INT UNSIGNED NOT NULL,
    PRIMARY KEY (lab_test_id),
    CONSTRAINT fk_hosp_lab_test_lab_test_id FOREIGN KEY (lab_test_id) REFERENCES
        lab_test (lab_test_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_hosp_lab_test_hosp_id FOREIGN KEY (hosp_id) REFERENCES
        hospitalisation (hosp_id) ON DELETE CASCADE ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structure for medical act
--

CREATE TABLE medical_act (
    med_act_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
    type ENUM('Χειρουργική', 'Διαγνωστική', 'Θεραπευτική') NOT NULL,
    med_proc_id INT UNSIGNED NOT NULL,
    start_datetime  DATETIME NOT NULL,
    end_datetime DATETIME NOT NULL,
    room_id INT UNSIGNED NOT NULL,
    cost  NUMERIC(8,2) NOT NULL DEFAULT 0.00 CHECK (cost >= 0),
    PRIMARY KEY (med_act_id),
    INDEX idx_fk_med_proc_id (med_proc_id),
    INDEX idx_fk_room_id (room_id),
    CHECK (start_datetime < end_datetime),
    CONSTRAINT fk_medical_act_med_procedure_id FOREIGN KEY (med_proc_id)
        REFERENCES medical_procedure (med_proc_id) ON DELETE RESTRICT ON
        UPDATE CASCADE,
    CONSTRAINT fk_medical_act_room_id FOREIGN KEY (room_id) REFERENCES room
        (room_id) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structure for surgical  act  and staff  assistance
--
```

```
CREATE TABLE surgical_act (
    med_act_id INT UNSIGNED NOT NULL,
    primary_doc_id VARCHAR(10) NOT NULL,
    PRIMARY KEY (med_act_id, primary_doc_id),
    CONSTRAINT fk_surgery_med_procedure_id FOREIGN KEY (med_act_id) REFERENCES
        medical_act (med_act_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_surgery_primary_doc_id FOREIGN KEY (primary_doc_id) REFERENCES
        doctor (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE surgical_act_doctor_assistants (
    med_act_id INT UNSIGNED NOT NULL,
    assistant_id  VARCHAR(10) NOT NULL,
    PRIMARY KEY (med_act_id, assistant_id),
    CONSTRAINT fk_surg_doc_surg_act_id FOREIGN KEY (med_act_id) REFERENCES
        surgical_act (med_act_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_surg_doc_assist_id FOREIGN KEY (assistant_id) REFERENCES doctor
        (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE surgical_act_nurse_assistants (
    med_act_id INT UNSIGNED NOT NULL,
    assistant_id  VARCHAR(10) NOT NULL,
    PRIMARY KEY (med_act_id, assistant_id),
    CONSTRAINT fk_surg_nurse_surg_act_id FOREIGN KEY (med_act_id) REFERENCES
        surgical_act (med_act_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_surg_nurse_assist_id FOREIGN KEY (assistant_id) REFERENCES nurse
        (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structure for relation  hospitalisation  <--< has >--- medical_act
--

CREATE TABLE hosp_med_act (
    med_act_id INT UNSIGNED NOT NULL,
    hosp_id INT UNSIGNED NOT NULL,
    PRIMARY KEY (med_act_id),
    INDEX idx_fk_hosp_id (hosp_id),
    CONSTRAINT fk_hosp_med_act_med_act_id FOREIGN KEY (med_act_id) REFERENCES
        medical_act (med_act_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_hosp_med_act_hospit_id FOREIGN KEY (hosp_id) REFERENCES
        hospitalisation (hosp_id) ON DELETE CASCADE ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structure for rating
--

CREATE TABLE rating (
    AMKA VARCHAR(45) NOT NULL,
    hosp_id INT UNSIGNED NOT NULL,
    medical_care TINYINT(1) UNSIGNED DEFAULT NULL CHECK (1 <= medical_care AND
        medical_care <= 5),
    nursing_care TINYINT(1) UNSIGNED DEFAULT NULL CHECK (1 <= nursing_care AND
        nursing_care <= 5),
```

```
    cleanliness  TINYINT(1) UNSIGNED DEFAULT NULL CHECK (1 <= cleanliness AND
        cleanliness <= 5),
    food TINYINT(1) UNSIGNED DEFAULT NULL CHECK (1 <= food AND food <= 5),
    experience TINYINT(1) UNSIGNED DEFAULT NULL CHECK (1 <= experience AND
        experience <= 5),
    PRIMARY KEY (AMKA, hosp_id),
    INDEX idx_fk_patient_id (AMKA),
    CONSTRAINT fk_rating_patient_id FOREIGN KEY (AMKA) REFERENCES patient
        (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_rating_hosp_id FOREIGN KEY (hosp_id) REFERENCES hospitalisation
        (hosp_id) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structures for pharmaceutical products
--

CREATE TABLE active_substance (
    act_sub_id  INT UNSIGNED NOT NULL AUTO_INCREMENT,
    act_sub_full  TEXT NOT NULL UNIQUE,
    PRIMARY KEY (act_sub_id),
    INDEX idx_act_sub (act_sub_full(100))
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE route_of_admission (
    route_id  INT UNSIGNED NOT NULL AUTO_INCREMENT,
    type VARCHAR(255) NOT NULL UNIQUE,
    PRIMARY KEY (route_id)
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE pharmaceutical_product (
    pharm_prod_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
    name VARCHAR(255) NOT NULL,
    auth_country VARCHAR(100) NOT NULL,
    marketing_auth_holder VARCHAR(255) NOT NULL,
    master_file_loc  VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL,
    phone VARCHAR(100) NOT NULL,
    PRIMARY KEY (pharm_prod_id)
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE product_act_sub (
    act_sub_id  INT UNSIGNED NOT NULL,
    pharm_prod_id INT UNSIGNED NOT NULL,
    PRIMARY KEY (act_sub_id, pharm_prod_id),
    CONSTRAINT fk_prod_act_sub_sub_id FOREIGN KEY (act_sub_id) REFERENCES
        active_substance (act_sub_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_prod_act_product_id FOREIGN KEY (pharm_prod_id) REFERENCES
        pharmaceutical_product (pharm_prod_id) ON DELETE RESTRICT ON UPDATE
        CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE product_route (
    route_id  INT UNSIGNED NOT NULL,
    pharm_prod_id INT UNSIGNED NOT NULL,
    PRIMARY KEY (route_id, pharm_prod_id),
    CONSTRAINT fk_prod_route_route_id FOREIGN KEY (route_id) REFERENCES
        route_of_admission (route_id) ON DELETE RESTRICT ON UPDATE CASCADE,
```

```
    CONSTRAINT fk_prod_route_product_id FOREIGN KEY (pharm_prod_id) REFERENCES
        pharmaceutical_product (pharm_prod_id) ON DELETE RESTRICT ON UPDATE
        CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structures for patient  allergies
--

CREATE TABLE patient_allergy (
    AMKA VARCHAR(10) NOT NULL,
    act_sub_id  INT UNSIGNED NOT NULL,
    PRIMARY KEY (AMKA, act_sub_id),
    CONSTRAINT fk_allergy_patient_id FOREIGN KEY (AMKA) REFERENCES patient
        (AMKA) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_allergy_substance_id FOREIGN KEY (act_sub_id) REFERENCES
        active_substance (act_sub_id) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structures for prescription
--

CREATE TABLE prescription (
    prescription_id  INT UNSIGNED NOT NULL AUTO_INCREMENT,
    doctor_id  VARCHAR(10) NOT NULL,
    patient_id  VARCHAR(10) NOT NULL,
    PRIMARY KEY (prescription_id),
    INDEX idx_fk_doctor_id (doctor_id),
    INDEX idx_fk_patient_id (patient_id),
    CONSTRAINT fk_prescription_doctor_id FOREIGN KEY (doctor_id) REFERENCES doctor
        (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_prescription_patient_id FOREIGN KEY (patient_id) REFERENCES
        patient (AMKA) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;


CREATE TABLE prescribed_products (
    prescription_id  INT UNSIGNED NOT NULL,
    pharm_prod_id INT UNSIGNED NOT NULL,
    start_date  DATE NOT NULL DEFAULT CURRENT_DATE,
    end_date DATE NOT NULL CHECK (end_date >= start_date),
    dosage VARCHAR(255) NOT NULL,
    frequency VARCHAR(100) NOT NULL,
    PRIMARY KEY (prescription_id, pharm_prod_id),
    CONSTRAINT unq_prescr_prod_presc_id_prod_id_sdate UNIQUE (prescription_id,
        pharm_prod_id, start_date),
    CONSTRAINT fk_prescr_prod_prescription_id FOREIGN KEY (prescription_id)
        REFERENCES prescription (prescription_id) ON DELETE RESTRICT ON UPDATE
        CASCADE,
    CONSTRAINT fk_prescr_prod_product_id FOREIGN KEY (pharm_prod_id) REFERENCES
        pharmaceutical_product (pharm_prod_id) ON DELETE RESTRICT ON UPDATE
        CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE hosp_prescription (
    hosp_id INT UNSIGNED NOT NULL,
    prescription_id  INT UNSIGNED NOT NULL,
```

```
    PRIMARY KEY (prescription_id),
    INDEX idx_fk_hosp_id (hosp_id),                                    -- Q10
    INDEX idx_fk_hosp_id_presc_id (hosp_id, prescription_id),          -- Q10
    CONSTRAINT fk_hosp_prescr_prescr_id FOREIGN KEY (prescription_id) REFERENCES
        prescription (prescription_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_hosp_prescr_hosp_id FOREIGN KEY (hosp_id) REFERENCES
        hospitalisation (hosp_id) ON DELETE RESTRICT ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Table structures for media
--

CREATE TABLE media (
    media_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
    path VARCHAR(255) NOT NULL,
    description  TEXT,
    PRIMARY KEY (media_id)
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE doctor_media (
    media_id INT UNSIGNED NOT NULL,
    doctor_id  VARCHAR(10) NOT NULL,
    PRIMARY KEY (media_id, doctor_id),
    CONSTRAINT fk_doctor_media_media_id FOREIGN KEY (media_id) REFERENCES media
        (media_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_doctor_media_entity_id FOREIGN KEY (doctor_id) REFERENCES doctor
        (AMKA) ON DELETE CASCADE ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE nurse_media (
    media_id INT UNSIGNED NOT NULL,
    nurse_id  VARCHAR(10) NOT NULL,
    PRIMARY KEY (media_id, nurse_id),
    CONSTRAINT fk_nurse_media_media_id FOREIGN KEY (media_id) REFERENCES media
        (media_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_nurse_media_entity_id FOREIGN KEY (nurse_id) REFERENCES nurse
        (AMKA) ON DELETE CASCADE ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;


CREATE TABLE admin_media (
    media_id INT UNSIGNED NOT NULL,
    admin_id VARCHAR(10) NOT NULL,
    PRIMARY KEY (media_id, admin_id),
    CONSTRAINT fk_admin_media_media_id FOREIGN KEY (media_id) REFERENCES media
        (media_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_admin_media_entity_id FOREIGN KEY (admin_id) REFERENCES
        administrative_staff (AMKA) ON DELETE CASCADE ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE equipment_media (
    media_id INT UNSIGNED NOT NULL,
    equipment_id VARCHAR(128) NOT NULL,
    PRIMARY KEY (media_id, equipment_id),
    CONSTRAINT fk_equipment_media_media_id FOREIGN KEY (media_id) REFERENCES
        media (media_id) ON DELETE CASCADE ON UPDATE CASCADE,
```

```
    CONSTRAINT fk_equipment_media_entity_id FOREIGN KEY (equipment_id)
        REFERENCES equipment (UID) ON DELETE CASCADE ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE bed_media (
    media_id INT UNSIGNED NOT NULL,
    bed_id INT UNSIGNED NOT NULL,
    PRIMARY KEY (media_id, bed_id),
    CONSTRAINT fk_bed_media_media_id FOREIGN KEY (media_id) REFERENCES media
        (media_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_bed_media_entity_id FOREIGN KEY (bed_id) REFERENCES bed
        (bed_id) ON DELETE CASCADE ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE room_media (
    media_id INT UNSIGNED NOT NULL,
    room_id INT UNSIGNED NOT NULL,
    PRIMARY KEY (media_id, room_id),
    CONSTRAINT fk_room_media_media_id FOREIGN KEY (media_id) REFERENCES media
        (media_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_room_media_entity_id FOREIGN KEY (room_id) REFERENCES room
        (room_id) ON DELETE CASCADE ON UPDATE CASCADE
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
    ============================================================================
--                                  Triggers
--
    ============================================================================

DELIMITER ;;

--
    ============================================================
--                      Doctor Supervison
--
    ============================================================

CREATE TRIGGER ins_doc_supervisor_cycle BEFORE INSERT ON doctor FOR EACH ROW
    BEGIN
    IF NEW.supervisor_id IS NOT NULL THEN

        SET @supervisee_id = NEW.AMKA;
        SET @supervisor_id = NEW.supervisor_id;
        SET @cycle_detection = 0;

        WITH RECURSIVE supervision_cycle AS (
            SELECT AMKA, supervisor_id
            FROM doctor
            WHERE AMKA = @supervisee_id
            UNION ALL
```

```
            SELECT doc.AMKA, doc.supervisor_id
            FROM doctor doc
            JOIN supervision_cycle hlpr ON doc.AMKA = hlpr.supervisor_id
        )
        SELECT COUNT(*) INTO @cycle_detection
        FROM supervision_cycle
        WHERE AMKA = @supervisor_id;

        IF @cycle_detection > 0 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Cyclic-doctor-supervision-is-not-permitted';
        END IF;
    END IF;
END;;

CREATE TRIGGER upd_doc_supervisor_cycle BEFORE UPDATE ON doctor FOR EACH
    ROW BEGIN
    IF NEW.supervisor_id IS NOT NULL THEN

        SET @supervisee_id = NEW.AMKA;
        SET @supervisor_id = NEW.supervisor_id;
        SET @cycle_detection = 0;

        WITH RECURSIVE supervision_cycle AS (
            SELECT AMKA, supervisor_id
            FROM doctor
            WHERE AMKA = @supervisee_id
            UNION ALL
            SELECT doc.AMKA, doc.supervisor_id
            FROM doctor doc
            JOIN supervision_cycle hlpr ON doc.AMKA = hlpr.supervisor_id
        )
        SELECT COUNT(*) INTO @cycle_detection
        FROM supervision_cycle
        WHERE AMKA = @supervisor_id;

        IF @cycle_detection > 0 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Cyclic-doctor-supervision-is-not-permitted';
        END IF;
    END IF;
END;;

--
-- Junior doctors must have a supervisor
-- Director cannot have a supervisor
--

CREATE TRIGGER ins_doc_supervisor BEFORE INSERT ON doctor FOR EACH ROW
    BEGIN
    DECLARE rank_t VARCHAR(45);

    IF NEW.rank = 'Διευθυντής' AND NEW.supervisor_id IS NOT NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Director-cannot-have-a-supervisor';
    END IF;

    IF NEW.rank = 'Ειδικευόμενος' AND NEW.supervisor_id IS NULL THEN
```

```
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Junior-doctors-must-have-a-supervisor';
    END IF;

    IF NEW.supervisor_id IS NOT NULL THEN
        SELECT rank INTO rank_t
        FROM doctor
        WHERE AMKA = NEW.supervisor_id;

        IF rank_t = 'Ειδικευόμενος' THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Junior-doctors-may-not-be-supervisors';
        END IF;
    END IF;
END;;


CREATE TRIGGER upd_doc_supervisor BEFORE UPDATE ON doctor FOR EACH ROW
    BEGIN
    DECLARE rank_t VARCHAR(45);

    IF NEW.rank = 'Διευθυντής' AND NEW.supervisor_id IS NOT NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Director-cannot-have-a-supervisor';
    END IF;

    IF NEW.rank = 'Ειδικευόμενος' AND NEW.supervisor_id IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Junior-doctors-must-have-a-supervisor';
    END IF;

    IF NEW.supervisor_id IS NOT NULL THEN
        SELECT rank INTO rank_t
        FROM doctor
        WHERE AMKA = NEW.supervisor_id;

        IF rank_t = 'Ειδικευόμενος' THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Junior-doctors-may-not-be-supervisors';
        END IF;
    END IF;
END;;

--
    ============================================================
--                      Hospitalisation
--
    ============================================================

CREATE TRIGGER upd_hospitalisation_discharge_date BEFORE UPDATE ON hospitalisation
    FOR EACH ROW BEGIN
    IF NEW.discharge_date IS NOT NULL
        AND NEW.admission_date > NEW.discharge_date THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Admission-date-must-precede-discharge-date';
    END IF;
END;;
```

```
CREATE TRIGGER ins_hospitalisation_triage AFTER INSERT ON hospitalisation FOR EACH
    ROW BEGIN
    UPDATE triage
    SET admission_time = NEW.admission_date
    WHERE triage_id = NEW.triage_id;
END;;

--
    ============================================================
--                          Shifts
--
    ============================================================


--
-- Shift verification  triggers
-- A shift may have status = 1 if and only if it  has
--          >= 3 doctors     AND
--          >= 6 nurses      AND
--          >= 2 admin staff
-- AND if a junior doctor is part of a shift ,
--    then at least  one senior doctor must be present

CREATE TRIGGER upd_shift_validity BEFORE UPDATE ON shift FOR EACH ROW BEGIN
    DECLARE d_cnt INT DEFAULT 0;
    DECLARE n_cnt INT DEFAULT 0;
    DECLARE a_cnt INT DEFAULT 0;
    DECLARE jr_cnt INT DEFAULT 0;
    DECLARE sr_cnt INT DEFAULT 0;

    IF NEW.status = TRUE THEN
        SELECT COUNT(*) INTO d_cnt
        FROM doctor_shift WHERE shift_id = NEW.shift_id;

        SELECT COUNT(*) INTO n_cnt
        FROM nurse_shift WHERE shift_id = NEW.shift_id;

        SELECT COUNT(*) INTO a_cnt
        FROM admin_shift WHERE shift_id = NEW.shift_id;

        IF (d_cnt < 3) OR (n_cnt < 6) OR (a_cnt < 2) THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Shift-does-not-meet-minimum-staffing-requirements';
        END IF;

        SELECT COUNT(*) INTO jr_cnt
        FROM doctor_shift ds
        INNER JOIN doctor d ON ds.AMKA = d.AMKA
        WHERE ds.shift_id = NEW.shift_id
          AND d.rank = 'Ειδικευόμενος';

        SELECT COUNT(*) INTO sr_cnt
        FROM doctor_shift ds
        INNER JOIN doctor d ON ds.AMKA = d.AMKA
        WHERE ds.shift_id = NEW.shift_id
          AND d.rank IN ('Επιμελητής-Α', 'Διευθυντής');

        IF jr_cnt  > 0 AND sr_cnt = 0 THEN
```

```
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Shift-with-junior-doctor-should-have-at-least-one-senior-doctor-
                assigned';
        END IF;

    END IF;
END;;

CREATE TRIGGER del_doctor_shift BEFORE DELETE ON doctor_shift FOR EACH ROW
    BEGIN
    DECLARE cnt INT DEFAULT 0;
    DECLARE jr_cnt INT DEFAULT 0;
    DECLARE sr_cnt INT DEFAULT 0;

    SELECT COUNT(*) INTO cnt
    from doctor_shift WHERE shift_id = OLD.shift_id;

    if  cnt <= 3 THEN
        UPDATE shift
        SET status = FALSE
        WHERE shift_id = OLD.shift_id;
    END IF;

    SELECT COUNT(*) INTO jr_cnt
    FROM doctor_shift ds
    INNER JOIN doctor d ON ds.AMKA = d.AMKA
    WHERE ds.shift_id = OLD.shift_id
        AND d.rank = 'Ειδικευόμενος';

    SELECT COUNT(*) INTO sr_cnt
    FROM doctor_shift ds
    INNER JOIN doctor d ON ds.AMKA = d.AMKA
    WHERE ds.shift_id = OLD.shift_id
        AND d.rank IN ('Επιμελητής-Α', 'Διευθυντής');

    IF jr_cnt  > 0 AND sr_cnt = 0 THEN
        UPDATE shift
        SET status = FALSE
        WHERE shift_id = OLD.shift_id;
    END IF;
END;;

CREATE TRIGGER del_nurse_shift BEFORE DELETE ON nurse_shift FOR EACH ROW
    BEGIN
    DECLARE cnt INT DEFAULT 0;

    SELECT COUNT(*) INTO cnt
    from nurse_shift WHERE shift_id = OLD.shift_id;

    if  cnt <= 6 THEN
        UPDATE shift
        SET status = FALSE
        WHERE shift_id = OLD.shift_id;
    END IF;
END;;

CREATE TRIGGER del_admin_shift BEFORE DELETE ON admin_shift FOR EACH ROW
    BEGIN
```

```
    DECLARE cnt INT DEFAULT 0;

    SELECT COUNT(*) INTO cnt
    from admin_shift WHERE shift_id = OLD.shift_id;

    if  cnt <= 2 THEN
        UPDATE shift
        SET status = FALSE
        WHERE shift_id = OLD.shift_id;
    END IF;
END;;

--
-- Additional restrictions
-- ~ no staff member may have 2 consecutive shifts
-- ~ no staff member may have > 3 consecutive night shifts
--

CREATE TRIGGER ins_consecutive_doctor_shift BEFORE INSERT ON doctor_shift FOR
    EACH ROW BEGIN
    DECLARE day_t DATE;
    DECLARE type_t INT;
    DECLARE cnt INT DEFAULT 0;

    SELECT day, type+0 INTO day_t, type_t
    FROM shift
    WHERE shift_id = NEW.shift_id;

    IF type_t = 3 THEN

        SELECT COUNT(*) INTO cnt
        FROM shift s
        INNER JOIN doctor_shift ds ON ds.shift_id = s.shift_id
        WHERE ds.AMKA = NEW.AMKA
          AND s.day >= DATE_SUB(day_t, INTERVAL 3 DAY)
          AND s.type+0 = 3;

        IF cnt >= 3 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Staff-member-cannot-be-assigned-to->-3-consecutive-shifts';
        END IF;
    END IF;

    IF EXISTS (
        SELECT *
        FROM doctor_shift ds
        INNER JOIN shift s ON ds.shift_id = s.shift_id
        WHERE ds.AMKA = NEW.AMKA
          AND (
                ( -- same day: previous of next shift exist
                    s.day = day_t
                    AND (s.type+0 = type_t - 1 OR s.type+0 = type_t + 1)
                )
            OR
                ( -- previous night and current morning
                    s.day = DATE_SUB(day_t, INTERVAL 1 DAY)
                    AND s.type+0 = 3
                    AND type_t = 1
```

```
            )
        OR
            ( -- current night and next morning
                s.day = DATE_ADD(day_t, INTERVAL 1 DAY)
                AND s.type = 1
                AND type_t = 3
            )
        )
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Staff-member-cannot-be-assigned-to-consecutive-shifts';
    END IF;
END;;

CREATE TRIGGER ins_consecutive_nurse_shift BEFORE INSERT ON nurse_shift FOR EACH
    ROW BEGIN
    DECLARE day_t DATE;
    DECLARE type_t INT;
    DECLARE cnt INT DEFAULT 0;

    SELECT day, type+0 INTO day_t, type_t
    FROM shift
    WHERE shift_id = NEW.shift_id;

    IF type_t = 3 THEN

        SELECT COUNT(*) INTO cnt
        FROM shift s
        INNER JOIN nurse_shift ds ON ds.shift_id = s.shift_id
        WHERE ds.AMKA = NEW.AMKA
          AND s.day >= DATE_SUB(day_t, INTERVAL 3 DAY)
          AND s.type+0 = 3;

        IF cnt >= 3 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Staff-member-cannot-be-assigned-to->-3-consecutive-shifts';
        END IF;
    END IF;

    IF EXISTS (
        SELECT *
        FROM nurse_shift ds
        INNER JOIN shift s ON ds.shift_id = s.shift_id
        WHERE ds.AMKA = NEW.AMKA
          AND (
              ( -- same day: previous of next shift exist
                  s.day = day_t
                  AND (s.type+0 = type_t - 1 OR s.type+0 = type_t + 1)
              )
          OR
              ( -- previous night and current morning
                  s.day = DATE_SUB(day_t, INTERVAL 1 DAY)
                  AND s.type+0 = 3
                  AND type_t = 1
              )
          OR
              ( -- current night and next morning
                  s.day = DATE_ADD(day_t, INTERVAL 1 DAY)
```

```
                AND s.type = 1
                AND type_t = 3
            )
        )
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Staff-member-cannot-be-assigned-to-consecutive-shifts';
    END IF;
END;;

CREATE TRIGGER ins_consecutive_admin_shift BEFORE INSERT ON admin_shift FOR EACH
    ROW BEGIN
    DECLARE day_t DATE;
    DECLARE type_t INT;
    DECLARE cnt INT DEFAULT 0;

    SELECT day, type+0 INTO day_t, type_t
    FROM shift
    WHERE shift_id = NEW.shift_id;

    IF type_t = 3 THEN

        SELECT COUNT(*) INTO cnt
        FROM shift s
        INNER JOIN admin_shift ds ON ds.shift_id = s.shift_id
        WHERE ds.AMKA = NEW.AMKA
          AND s.day >= DATE_SUB(day_t, INTERVAL 3 DAY)
          AND s.type+0 = 3;

        IF cnt >= 3 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Staff-member-cannot-be-assigned-to->-3-consecutive-shifts';
        END IF;
    END IF;

    IF EXISTS (
        SELECT *
        FROM admin_shift ds
        INNER JOIN shift s ON ds.shift_id = s.shift_id
        WHERE ds.AMKA = NEW.AMKA
          AND (
              ( -- same day: previous of next shift exist
                  s.day = day_t
                  AND (s.type+0 = type_t - 1 OR s.type+0 = type_t + 1)
              )
            OR
              ( -- previous night and current morning
                  s.day = DATE_SUB(day_t, INTERVAL 1 DAY)
                  AND s.type+0 = 3
                  AND type_t = 1
              )
            OR
              ( -- current night and next morning
                  s.day = DATE_ADD(day_t, INTERVAL 1 DAY)
                  AND s.type = 1
                  AND type_t = 3
              )
          )
```

```
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Staff-member-cannot-be-assigned-to-consecutive-shifts';
    END IF;
END;;

--
-- Monthly shift amount limit
-- ~ doctors: 15
-- ~ nurses: 20
-- ~ admin: 25
--

CREATE TRIGGER ins_doctor_monthly_shift_lim BEFORE INSERT ON doctor_shift FOR
    EACH ROW BEGIN
    DECLARE date_t DATE;
    DECLARE cnt INT DEFAULT 0;

    SELECT day INTO date_t
    FROM shift
    WHERE shift_id = NEW.shift_id;

    SELECT COUNT(*) INTO cnt
    FROM doctor_shift ds
    INNER JOIN shift s ON ds.shift_id = s.shift_id
    WHERE ds.AMKA = NEW.AMKA
      AND YEAR(s.day) = YEAR(date_t)
      AND MONTH(s.day) = MONTH(date_t);

    IF cnt >= 15 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Doctors-may-be-assigned-to-a-maximum-of-15-shifts-per-month';
    END IF;
END;;

CREATE TRIGGER ins_nurse_monthly_shift_lim BEFORE INSERT ON nurse_shift FOR EACH
    ROW BEGIN
    DECLARE date_t DATE;
    DECLARE cnt INT DEFAULT 0;

    SELECT day INTO date_t
    FROM shift
    WHERE shift_id = NEW.shift_id;

    SELECT COUNT(*) INTO cnt
    FROM nurse_shift ds
    INNER JOIN shift s ON ds.shift_id = s.shift_id
    WHERE ds.AMKA = NEW.AMKA
      AND YEAR(s.day) = YEAR(date_t)
      AND MONTH(s.day) = MONTH(date_t);

    IF cnt >= 20 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Nurses-may-be-assigned-to-a-maximum-of-20-shifts-per-month';
    END IF;
END;;
```

```
CREATE TRIGGER ins_admin_monthly_shift_lim BEFORE INSERT ON admin_shift FOR
    EACH ROW BEGIN
    DECLARE date_t DATE;
    DECLARE cnt INT DEFAULT 0;

    SELECT day INTO date_t
    FROM shift
    WHERE shift_id = NEW.shift_id;

    SELECT COUNT(*) INTO cnt
    FROM admin_shift ds
    INNER JOIN shift s ON ds.shift_id = s.shift_id
    WHERE ds.AMKA = NEW.AMKA
      AND YEAR(s.day) = YEAR(date_t)
      AND MONTH(s.day) = MONTH(date_t);

    IF cnt >= 25 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'administrative-staff-may-be-assigned-to-a-maximum-of-25-shifts-per-
            month';
    END IF;
END;;


--
    ============================================================
--                          Prescriptions
--
    ============================================================

CREATE TRIGGER ins_prescribed_prod_patient_allergy BEFORE INSERT ON
    prescribed_products FOR EACH ROW BEGIN
    DECLARE patient_id_t VARCHAR(10);
    DECLARE act_sub_id_t INT UNSIGNED;

    SELECT patient_id INTO patient_id_t
    FROM prescription
    WHERE prescription_id = NEW.prescription_id;

    IF EXISTS (
        SELECT *
        FROM product_act_sub pas
        INNER JOIN patient_allergy pa ON pas.act_sub_id = pa.act_sub_id
        WHERE pa.AMKA = patient_id_t
          AND pas.pharm_prod_id = NEW.pharm_prod_id
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Patient-allergic-to-prescribed-active-substance';
    END IF;
END;;



CREATE TRIGGER upd_prescribed_prod_patient_allergy BEFORE UPDATE ON
    prescribed_products FOR EACH ROW BEGIN
    DECLARE patient_id_t VARCHAR(10);

    SELECT patient_id INTO patient_id_t
```

```
    FROM prescription
    WHERE prescription_id = NEW.prescription_id;

    IF EXISTS (
        SELECT *
        FROM product_act_sub pas
        INNER JOIN patient_allergy pa ON pas.act_sub_id = pa.act_sub_id
        WHERE pa.AMKA = patient_id_t
          AND pas.pharm_prod_id = NEW.pharm_prod_id
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Patient-allergic-to-prescribed-active-substance';
    END IF;
END;;

--
    ============================================================
--                          Surgery
--
    ============================================================

--
-- Prevent two acts from taking place at the same place at the same time
--

CREATE TRIGGER ins_surgery_locality_overlap BEFORE INSERT ON surgical_act FOR EACH
    ROW BEGIN
    DECLARE start_t DATETIME;
    DECLARE end_t TIME;
    DECLARE room_t INT UNSIGNED;

    SELECT start_datetime, end_datetime, room_id INTO start_t, end_t, room_t
    FROM medical_act
    WHERE med_act_id = NEW.med_act_id;

    IF EXISTS (
        SELECT *
        FROM surgical_act sa
        INNER JOIN medical_act ma ON ma.med_act_id = sa.med_act_id
        WHERE ma.room_id = room_t
        AND (ma.end_datetime > start_t AND ma.start_datetime < end_t)
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Room-in-use';
    END IF;
END;;

--
-- Prevent temporal overlaps when assigning staff
--

CREATE TRIGGER ins_surgery_temporality_main_doc BEFORE INSERT ON surgical_act FOR
    EACH ROW BEGIN
    DECLARE start_t DATETIME;
    DECLARE end_t TIME;

    SELECT start_datetime, end_datetime INTO start_t, end_t
    FROM medical_act
```

```
    WHERE med_act_id = NEW.med_act_id;

    IF EXISTS (
        SELECT *
        FROM surgical_act sa
        INNER JOIN medical_act ma ON ma.med_act_id = sa.med_act_id
        WHERE sa.primary_doc_id = NEW.primary_doc_id
        AND (ma.end_datetime > start_t AND ma.start_datetime < end_t)
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Primary-doctor-already-assigned-at-given-time';
    END IF;
END;;

CREATE TRIGGER ins_surgery_temporality_assist_doc BEFORE INSERT ON
    surgical_act_doctor_assistants FOR EACH ROW BEGIN
    DECLARE start_t DATETIME;
    DECLARE end_t TIME;
    DECLARE primary_doc_t VARCHAR(10);

    SELECT start_datetime, end_datetime INTO start_t, end_t
    FROM medical_act
    WHERE med_act_id = NEW.med_act_id;

    IF EXISTS (
        SELECT *
        FROM surgical_act_doctor_assistants sa
        INNER JOIN medical_act ma ON ma.med_act_id = sa.med_act_id
        WHERE sa.assistant_id = NEW.assistant_id
        AND (ma.end_datetime > start_t AND ma.start_datetime < end_t)
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Assistant-doctor-already-assigned-at-given-time';
    END IF;


    -- Prevent insertion if doctor is already primary doctor of act
    SELECT primary_doc_id INTO primary_doc_t
    FROM surgical_act
    WHERE med_act_id = NEW.med_act_id;

    IF primary_doc_t = NEW.assistant_id THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Assistant-doctor-already-assigned-as-primary-doctor';
    END IF;
END;;

CREATE TRIGGER ins_surgery_temporality_assist_nurse BEFORE INSERT ON
    surgical_act_nurse_assistants FOR EACH ROW BEGIN
    DECLARE start_t DATETIME;
    DECLARE end_t TIME;

    SELECT start_datetime, end_datetime INTO start_t, end_t
    FROM medical_act
    WHERE med_act_id = NEW.med_act_id;

    IF EXISTS (
        SELECT *
```

```
        FROM surgical_act_nurse_assistants sa
        INNER JOIN medical_act ma ON ma.med_act_id = sa.med_act_id
        WHERE sa.assistant_id = NEW.assistant_id
        AND (ma.end_datetime > start_t AND ma.start_datetime < end_t)
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Assistant-nurse-doctor-already-assigned-at-given-time';
    END IF;
END;;

--
    ============================================================
--                          Rating
--
    ============================================================
-- A rating can only be given after a hospitalisation  has ended

CREATE TRIGGER ins_rating BEFORE INSERT ON rating FOR EACH ROW BEGIN
    DECLARE discharge_t TIMESTAMP;

    SELECT discharge_date INTO discharge_t
    FROM hospitalisation
    WHERE hosp_id = NEW.hosp_id;

    IF discharge_t  IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Hospitalisation-in-progress;-cannot-assign-rating';
    END IF;
END;;

--
    ============================================================
--                      Medical/test procedures
--
    ============================================================

CREATE TRIGGER ins_lab_test BEFORE INSERT ON lab_test FOR EACH ROW BEGIN
    IF NEW.med_proc_id <= 6608 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Lab-test-are-described-by-sections-Γ,-Δ,-E-of-the-Greek-medical-
            procedure-encoding-standard';
    END IF;
END;;

CREATE TRIGGER upd_lab_test BEFORE UPDATE ON lab_test FOR EACH ROW BEGIN
    IF NEW.med_proc_id <= 6608 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Lab-test-are-described-by-sections-Γ,-Δ,-E-of-the-Greek-medical-
            procedure-encoding-standard';
    END IF;
END;;

CREATE TRIGGER ins_medical_act BEFORE INSERT ON medical_act FOR EACH ROW
    BEGIN
    IF NEW.med_proc_id > 6608 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Medical-acts-are-described-by-sections-A,-B-of-the-Greek-medical-
            procedure-encoding-standard';
```

```
    END IF;
END;;

CREATE TRIGGER upd_medical_act BEFORE UPDATE ON medical_act FOR EACH ROW
    BEGIN
    IF NEW.med_proc_id > 6608 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Medical-acts-are-described-by-sections-A,-B-of-the-Greek-medical-
            procedure-encoding-standard';
    END IF;
END;;

DELIMITER ;

--
    ========================================================================
--                                  Views
--
    ========================================================================

--
-- Triage: patients treated based on emergency level + FIFO
--
CREATE VIEW vw_triage_queue AS
SELECT *
FROM triage
WHERE admission_time IS NULL
ORDER BY level, arrival_time ASC;

--
-- Print doctor specialisation along with values of doctor table
--
CREATE VIEW vw_doctor_info AS
SELECT d.AMKA, d.first_name, d.last_name, d.middle_name, d.date_of_birth, d.date_of_employment,
    d.license_number, d.supervisor_id, s.spec_code, s.description_grc
FROM doctor d
INNER JOIN doc_spec ds ON d.AMKA = ds.AMKA
INNER JOIN specialisation s ON ds.spec_code = s.spec_code;



--
    ========================================================================
--                                  Functions
--
    ========================================================================

DELIMITER ;;

--
-- Calculate new cost, depending on stay duration
--

CREATE FUNCTION calculate_hospit_cost(hosp_id_t INT)
RETURNS NUMERIC(15, 2)
DETERMINISTIC
READS SQL DATA
BEGIN
```

```
    DECLARE base_cost_t NUMERIC(9, 2);
    DECLARE mean_time_t INT UNSIGNED;
    DECLARE admission_t TIMESTAMP;
    DECLARE discharge_t TIMESTAMP;
    DECLARE elapsed_days INT DEFAULT 0;
    DECLARE extra_days INT DEFAULT 0;

    SELECT c.base_cost, c.mean_hospit_time, h.admission_date, h.discharge_date
    INTO base_cost_t, mean_time_t, admission_t, discharge_t
    FROM hospitalisation h
    INNER JOIN costing c ON h.costing_id = c.costing_id
    WHERE h.hosp_id = hosp_id_t;

    IF discharge_t  IS NULL THEN
        SET discharge_t = NOW();
    END IF;

    SET elapsed_days = TIMESTAMPDIFF(DAY, admission_t, discharge_t);

    IF elapsed_days <= mean_time_t THEN
        RETURN base_cost_t;
    END IF;

    SET extra_days = elapsed_days − mean_time_t;

    RETURN base_cost_t ∗ (1 + 0.1 ∗ extra_days);

END;;

DELIMITER ;

SET SQL_MODE=@OLD_SQL_MODE;
SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS;
SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS;
```

References

[1] gesy. *GESY Doctor Specialisation*. URL: https://www.gesy.org.cy/el-gr/annualreport/042026-specialty-restrictions-ahp-mp.xlsx (visited on 04/2026).

[2] GS1. *GS1 Healthcare Standards*. URL: https://www.gs1.org/industries/healthcare/standards (visited on 04/2026).

[3] MariaDB Foundation. *MariaDB Documentation*. URL: https://mariadb.com/docs/ (visited on 04/2026).

[4] Oracle Corporation. *MySQL Documentation*. URL: https://dev.mysql.com/doc/ (visited on 04/2026).

[5] Abraham Silberschatz, Henry F. Korth, and S. Sudarshan. *Database System Concepts*. 7th. McGraw-Hill, 2020. URL: https://www.db-book.com/.

[6] Stack Overflow. *How do I enforce a total participation constraint on 1:N relationship*. URL: https://stackoverflow.com/questions/65661279/how-do-i-enforce-a-total-participation-constraint-on-1n-relationship-in-a-relat (visited on 04/2026).

[7] Stack Overflow. *How to implement one-to-one, one-to-many and many-to-many relationships*. URL: https://stackoverflow.com/questions/7296846/how-to-implement-one-to-one-one-to-many-and-many-to-many-relationships-while-de (visited on 04/2026).

[8] Stack Overflow. *How to write a trigger to abort delete in MySQL*. URL: https://stackoverflow.com/questions/7595714/how-to-write-a-trigger-to-abort-delete-in-mysql (visited on 04/2026).

[9] Stack Overflow. *MySQL collation utf8mb4 unicode ci vs default collation*. URL: https://stackoverflow.com/questions/51278467/mysql-collation-utf8mb4-unicode-ci-vs-utf8mb4-default-collation (visited on 04/2026).